

Tech & Teach

Digital Chip Design Program

RTL Design

Mohammedalnaif

Mohammedalkhalifah

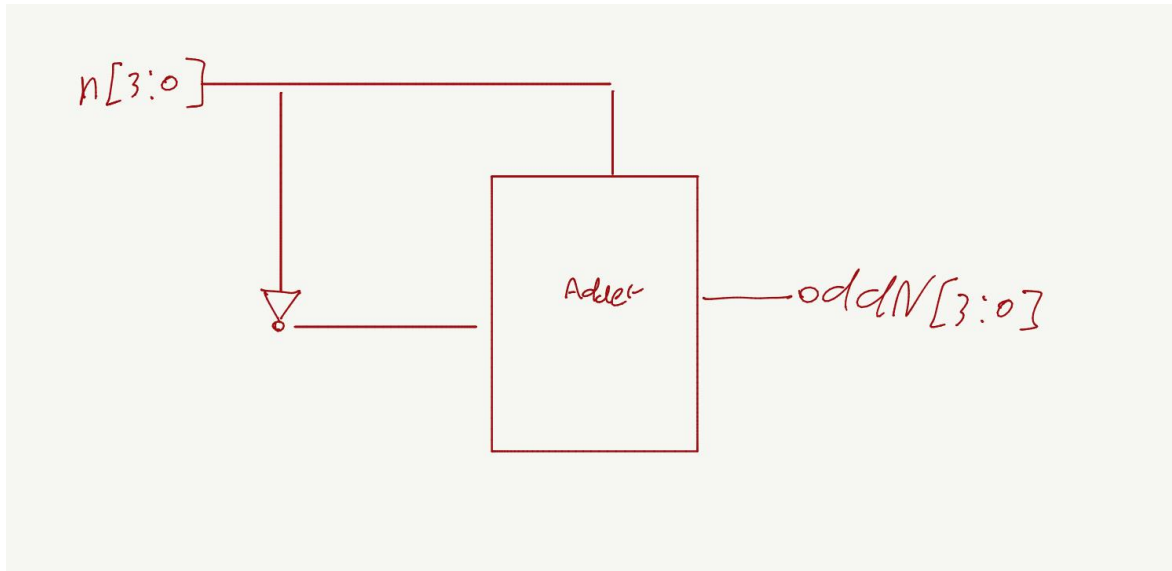
Task #06

9/7/2026

Combinational Datapath design Part 2

Tech & Teach

Task #6.1

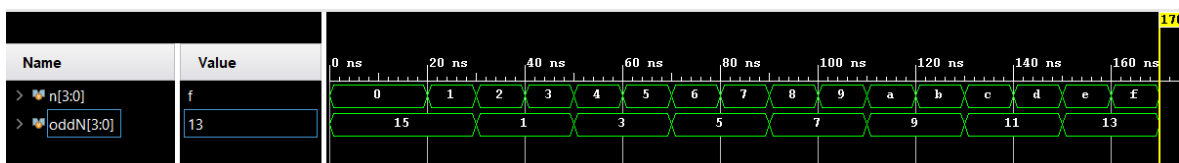


```

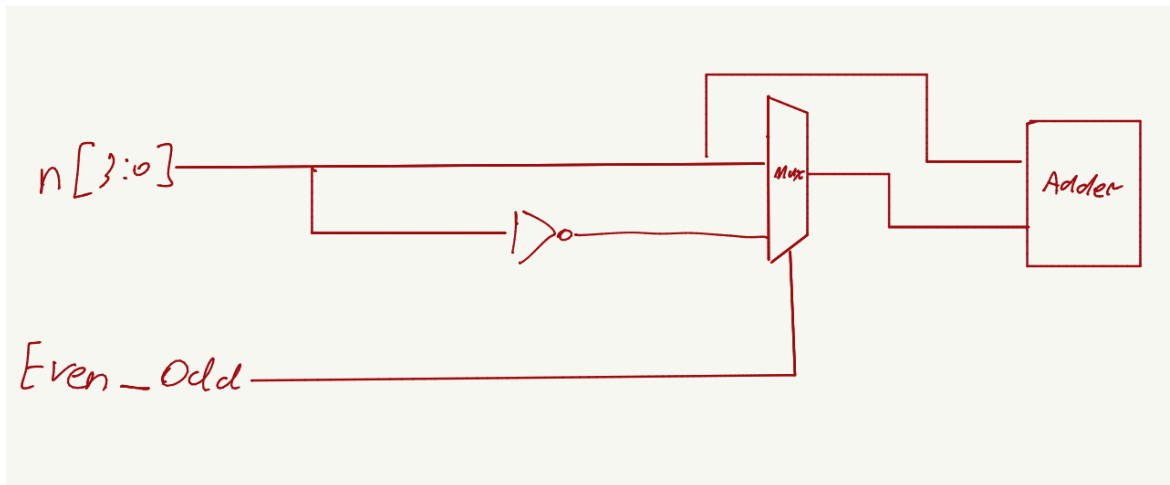
module toOdd (
input logic [3:0] n, output logic [3:0] oddN );
always_comb begin oddN = n + (~n[0]);
end
endmodule

`timescale 1ns / 1ps
module toOdd_tb();
    logic [3:0] n;
    logic [3:0] oddN;
    toOdd dut (
        .n(n), .oddN(oddN) );
    initial begin n = 4'b0000; #10;
    for (int i = 0; i < 16; i++) begin
        n = i;
        #10;
    end
    $finish;
end
endmodule

```



Task #6.1.2



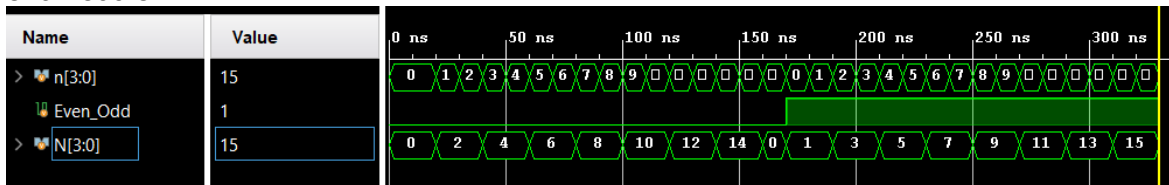
```
module num_convertor
( input logic [3:0] n,
  input logic Even_Odd,
  output logic [3:0] N
);
logic add_val;
always_comb begin
  if (Even_Odd == 1'b0)
    add_val = n[0];
  else
    add_val = ~n[0];
  N = n + add_val;
end
endmodule
```

```
`timescale 1ns / 1ps
module num_convertor_tb();
logic [3:0] n;
logic Even_Odd;
logic [3:0] N;
num_convertor dut ( .n(n),
  .Even_Odd(Even_Odd),
  .N(N)
);
initial begin n = 4'b0000;
  Even_Odd = 1'b0; #10;
```

```

Even_Odd = 1'b0;
for (int i = 0; i < 16; i++) begin
    n = i;
    #10;
End
$finish;
end
endmodule

```



Task #6.2

```

module isspecial(
    input logic [2:0] number,
    output logic      special
);

    logic [2:0] w;

```

```

    comparator comp0(
        .A(number),
        .B(3'd3),
        .AeqB(w[0]),
        .AgtB(),
        .AltB()
    );

```

```

    comparator comp1(
        .A(number),

```

```

.B(3'd5),    // Fixed: Explicit 3-bit width
.AeqB(w[1]),
.AgtB(),
.AltB()
);

```

```

comparator comp2(

```

```

.A(number),

```

```

.B(3'd7),

```

```

.AeqB(w[2]),

```

```

.AgtB(),

```

```

.AltB()

```

```

);

```

```

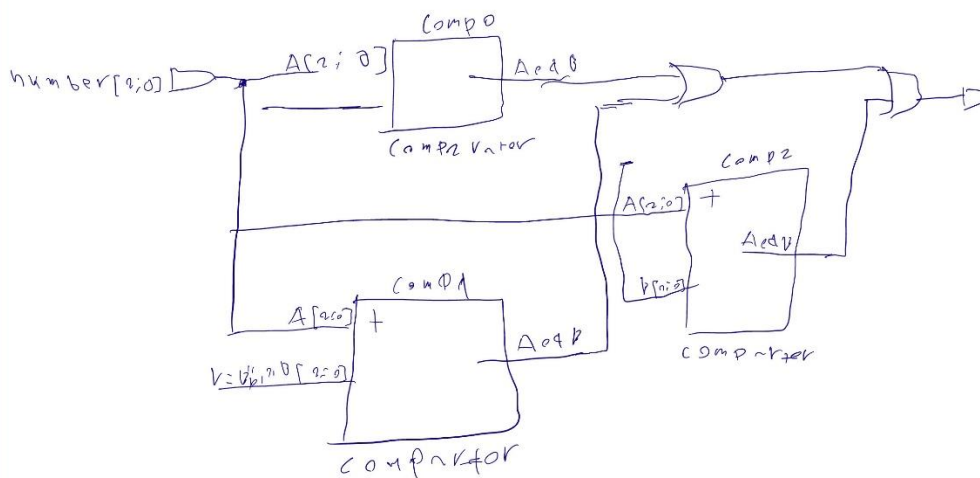
or (special, w[0], w[1], w[2]);

```

```

endmodule

```



```
module isspecial_tb;

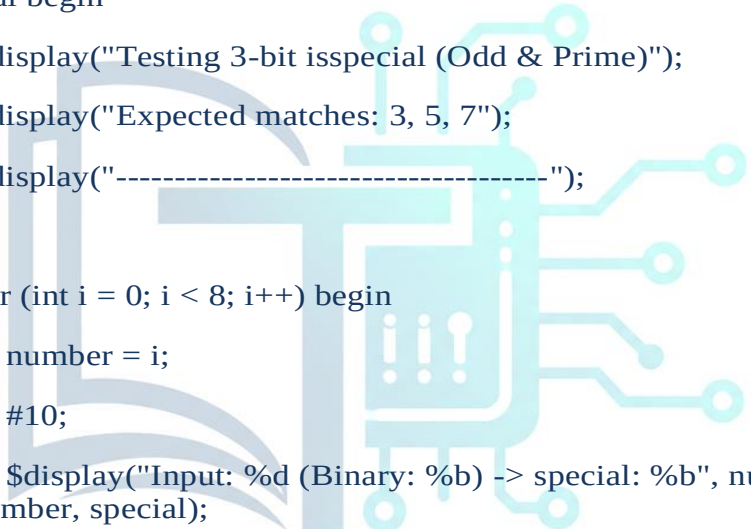
    logic [2:0] number;
    logic      special;

    isspecial uut (
        .number(number),
        .special(special)
    );

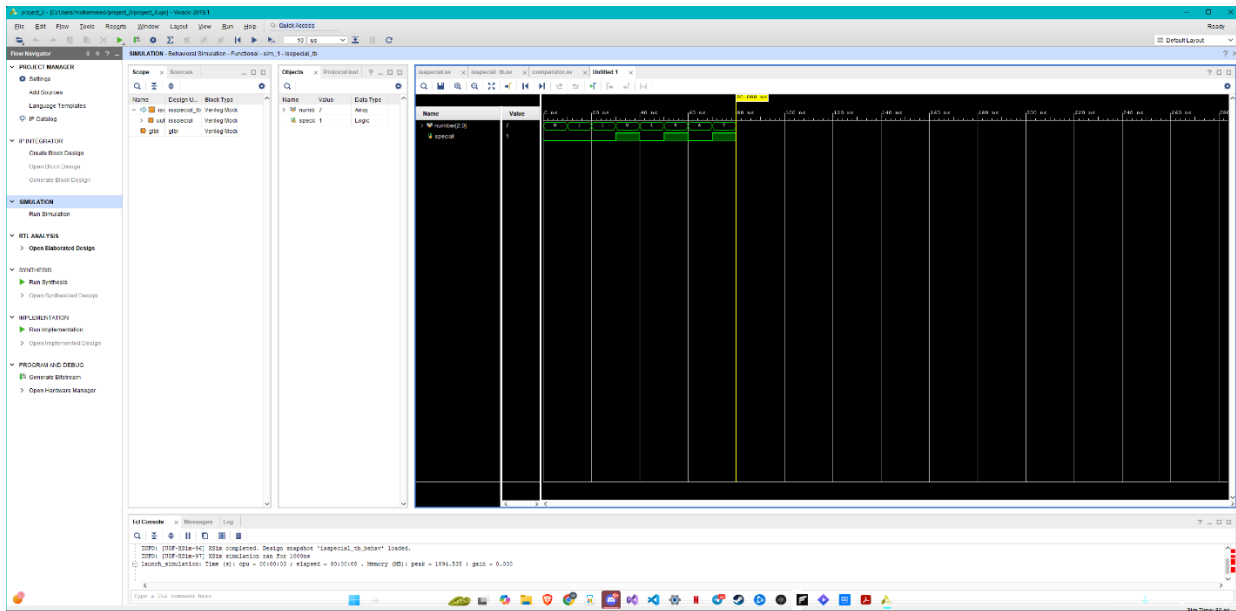
    initial begin
        $display("Testing 3-bit isspecial (Odd & Prime)");
        $display("Expected matches: 3, 5, 7");
        $display("-----");

        for (int i = 0; i < 8; i++) begin
            number = i;
            #10;
            $display("Input: %d (Binary: %b) -> special: %b", number,
                number, special);
        end

        $finish;
    end
endmodule
```



Tech & Teach



Task #6.3

```

module CaesarCipher (
    input logic [6:0] p,
    output logic [6:0] c
);

```

```

    logic [6:0] x1, x2, x3, x4;
    logic lt25;

```

```

    AdderSubtractor U1 (

```

```
        .A(p),
```

```
        .B(7'd65),
```

```
        .M(1'b1),
```

```
        .S(x1),
```

```
        .Cout()  
    );
```

AdderSubtractor U2 (

```
    .A(x1),  
    .B(7'd3),  
    .M(1'b0),  
    .S(x2),  
    .Cout()  
);
```

Comparator U3 (

```
    .A(7'd25),  
    .B(x2),  
    .AgtB(),  
    .AeqB(),  
    .AltB(lt25)  
);
```

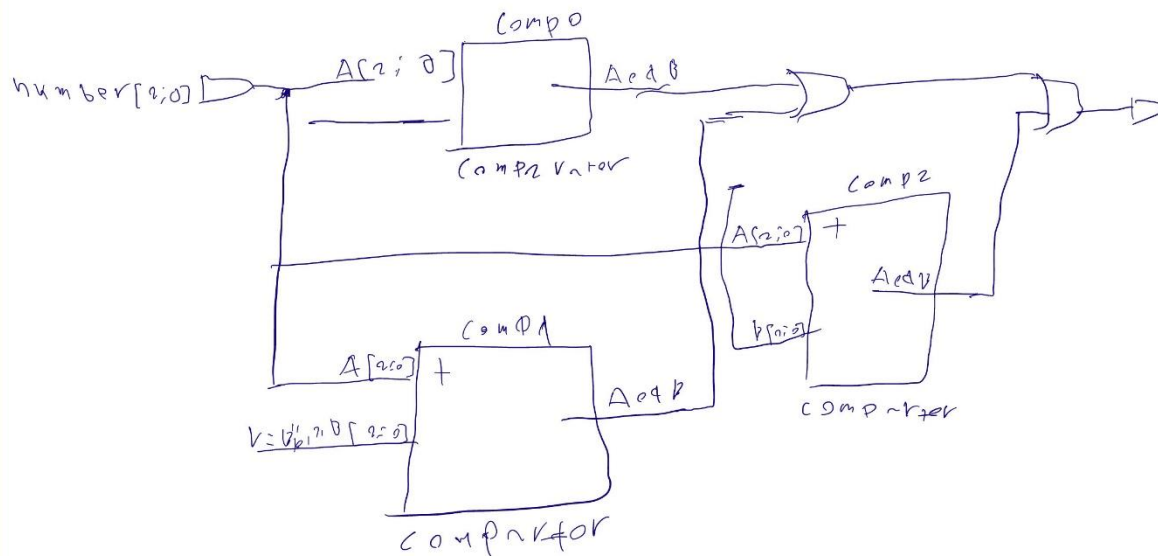
AdderSubtractor U4 (

```
.A(x2),  
.B(7'd26),  
.M(1'b1),  
.S(x3),  
.Cout()  
);
```

```
mux2x1 U5 (  
.I0(x2),  
.I1(x3),  
.Sel(1t25),  
.Y(x4)  
);
```

```
AdderSubtractor U6 (  
.A(x4),  
.B(7'd65),  
.M(1'b0),  
.S(c),  
.Cout()  
);
```

Endmodule



```
module CaesarCipher_tb;
```

```
    logic [6:0] p;
```

```
    logic [6:0] c;
```

```
    // Exact case match to your structural
    module
```

```
        CaesarCipher uut (
```

```
            .p(p),
```

```
            .c(c)
```

```
        );
```

```
    initial begin
```

```
$display("Testing Caesar Cipher  
Hardware (Uppercase A-Z)");
```

```
$display("-----  
----");
```

```
// Test uppercase 'A' (65) -> should be 'D'  
(68)
```

```
p = 7'd65; #10;
```

```
$display("Input: %c (%d) -> Output: %c  
(%d)", p, p, c, c);
```

```
// Test uppercase 'X' (88) -> should wrap  
around to 'A' (65)
```

```
p = 7'd88; #10;
```

```
$display("Input: %c (%d) -> Output: %c  
(%d)", p, p, c, c);
```

```
// Test uppercase 'Z' (90) -> should wrap  
around to 'C' (67)
```

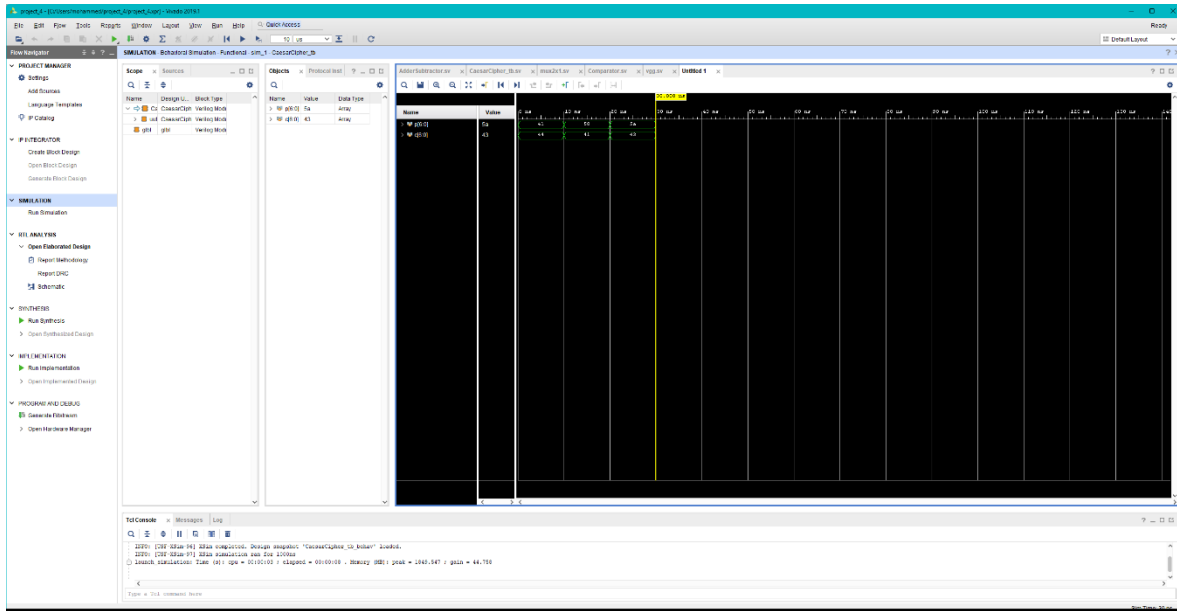
```
p = 7'd90; #10;
```

```
$display("Input: %c (%d) -> Output: %c  
(%d)", p, p, c, c);
```

```
$finish;
```

```
end
```

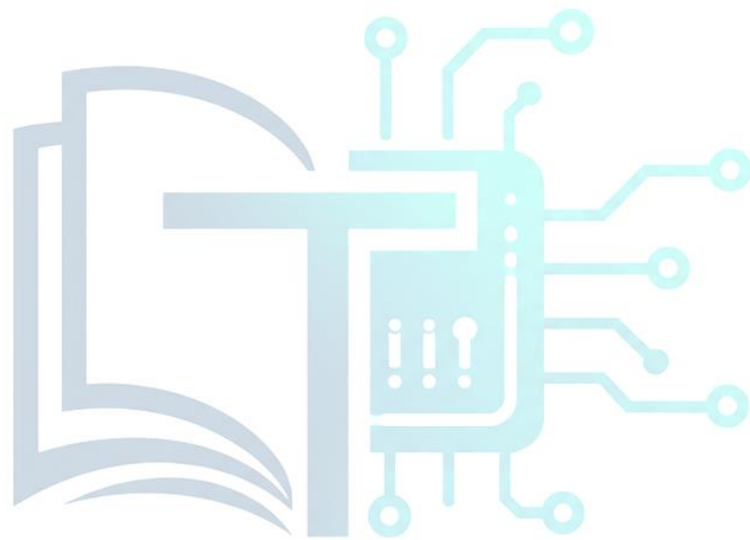
```
endmodule
```



Work distribution

Section	Abdulmalek	Omar	Khaled
Report	✓		
Task n.x		✓	
Task n.x+1			✓

--	--	--	--



Tech & Teach